ITM Business School
ITM SKILLS UNIVERSITY

SCHOOL OF FUTURE TECH
ITM Skills University

ITM SKILLS UNIVERSITY

B.Tech Computer Science Engineering

WORKHUB

A Co-working Space Booker Platform using
Node.js, Express.js & MongoDB
Case Study 60

Submitted in Partial Fulfilment of the Requirements for the Course:

Backend Development – Node.js, Express.js & MongoDB

1. Student Details

Submitted By:

Name	Student ID
Raj Bharat Rasal	150096725066

Programme: B.Tech Computer Science Engineering

Institution: ITM Skills University

Academic Year: 2025–2029

Contents

1. Student Details 1
2. Problem Statement and Project Objectives 2
3. System Architecture and Workflow 4
4. Database Design 6
5. Backend Implementation 8
6. Frontend Overview 13
7. Execution Screenshots 14
8. Results and Observations 15
9. Conclusion and Future Scope 16

2. Problem Statement and Project Objectives

2.1 Problem Statement

Flexible workspaces need hot-desk reservations. The task (Case Study 60) is to build the backend for a Co-working Space Booker called **WorkHub** using Node.js, Express.js, MongoDB, Firebase and Socket.io.

The API must let a co-working space run its daily operations:

- Members book desks and meeting rooms by time and location.
- Utilization is tracked.
- Members invite guests.
- Amenities (printer, parking) can be booked.
- Monthly passes are supported.
- Space managers approve bookings and see revenue reports.
- Socket.io delivers real-time availability updates.
- Firebase push notifications confirm bookings.

2.2 Why This Project?

A booking system has one rule that is easy to state and hard to enforce: **two people must never hold the same desk at the same time**. When many members book at once, an application that first checks availability and then saves can let both requests through. Network drops add a second problem: a member who retries a payment may be charged twice. WorkHub solves these problems on the server, at the database and API level, rather than hoping clients behave correctly.

Proposed solution. A REST API on Node.js and Express.js stores members, spaces, bookings, passes, guests and transactions in MongoDB through Mongoose. JWT protects restricted routes, Socket.IO pushes live availability, and three server-side safeguards guard the critical operations: a unique index that rejects double bookings, an Idempotency-Key that makes retries safe, and a signed QR token for check-in. Figure 1 shows the complete example journey from the brief.

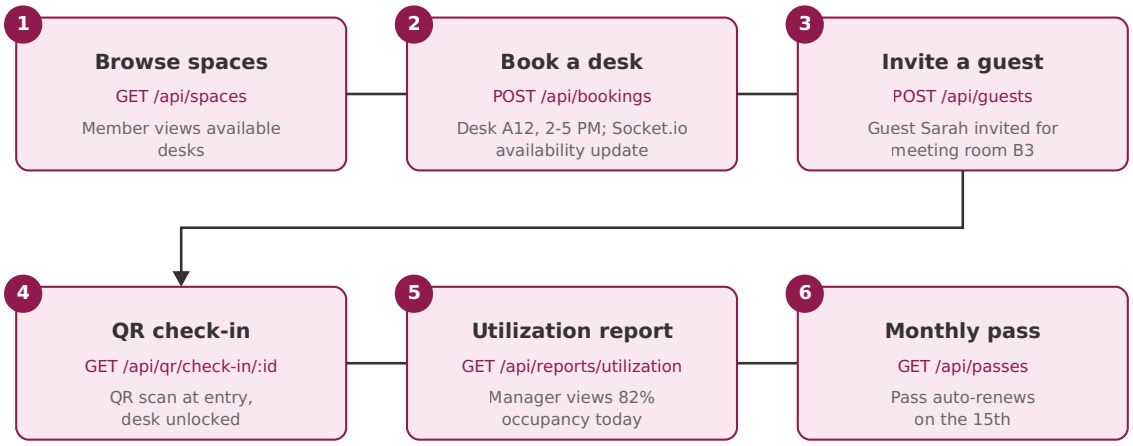


Figure 1: Example user flow from the case study

2.3 Objectives

No.	Objective	Description
1	Authentication and roles	JWT-based authentication, with member and manager roles enforced on routes
2	Space and booking management	Members book desks and meeting rooms by time and location, with full CRUD
3	Conflict-free reservations	The same slot can never be booked twice, even under simultaneous requests
4	Safe retries	A repeated checkout request does not charge or book twice
5	Real-time availability	Socket.io pushes availability changes to every connected client
6	Passes, guests and amenities	Monthly passes, guest invitations and amenity bookings
7	QR check-in	A signed QR token unlocks the booked desk on arrival
8	Manager analytics	Utilization and revenue reports built with aggregation pipelines
9	Notifications	Firebase push notifications for booking confirmations

3. System Architecture and Workflow

3.1 System Workflow

The platform serves two kinds of users. A **member** browses spaces, books, invites guests and checks in. A **manager** adds spaces and reads the reports. The numbered journey in Figure 1 maps each step to its endpoint.

Step	Actor	Endpoint	What happens
1	Member	GET /api/spaces	Views the available desks
2	Member	POST /api/bookings	Books Desk A12 for 2 to 5 PM; Socket.io pushes the availability update
3	Member	POST /api/guests	Invites guest Sarah for meeting room B3
4	Member	GET /api/qr/check-in/:id	Scans the QR code at the entry and the desk unlocks
5	Manager	GET /api/reports/utilization	Views 82% occupancy today
6	Member	GET /api/passes	Monthly pass auto-renews on the 15th

3.2 System Architecture

The system is organised as a monorepo with two applications: a **backend** REST and WebSocket API, and a **frontend** single page application. Figure 2 shows how a request travels. The browser talks to Express over REST for data and mutations, and over Socket.IO for live updates. Express authenticates, authorises and validates each request, then uses Mongoose to read and write MongoDB.

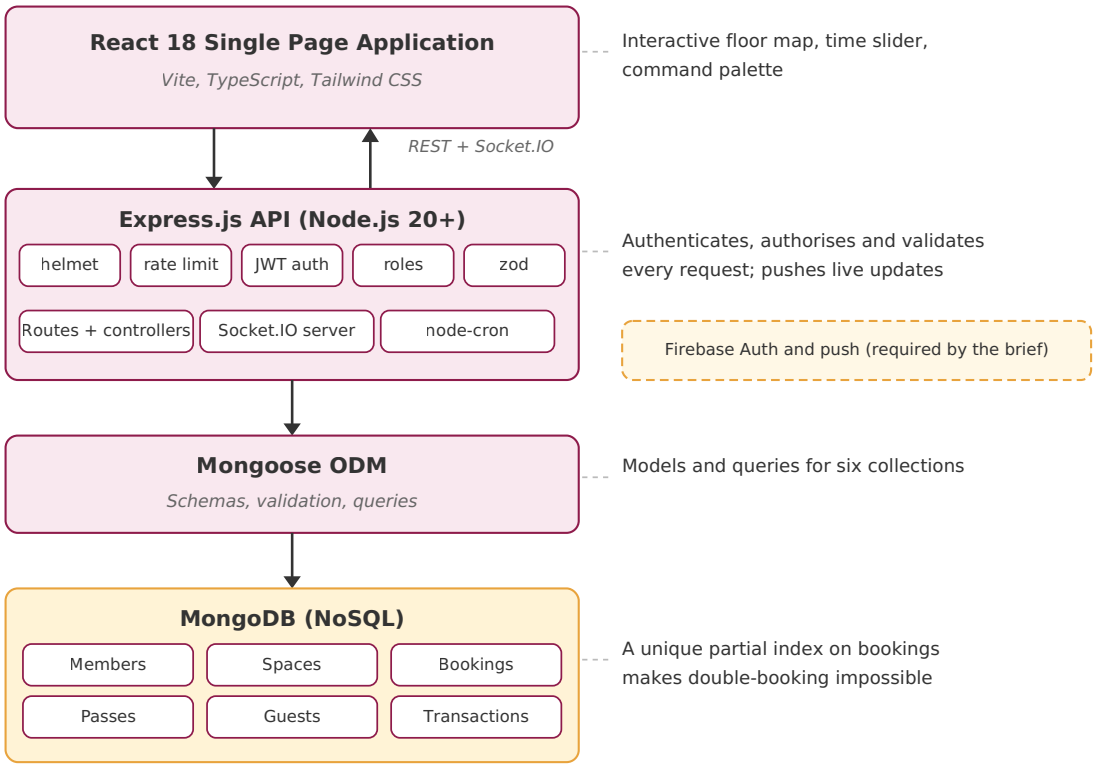


Figure 2: Layered architecture of WorkHub

3.3 Request Pipeline

Every protected request passes through the same layers before reaching a controller (Figure 3). Security hardening (helmet, rate limiting) applies to all traffic. Authentication, role authorization and payload validation protect each route. A request that fails a layer stops there with an appropriate error code.

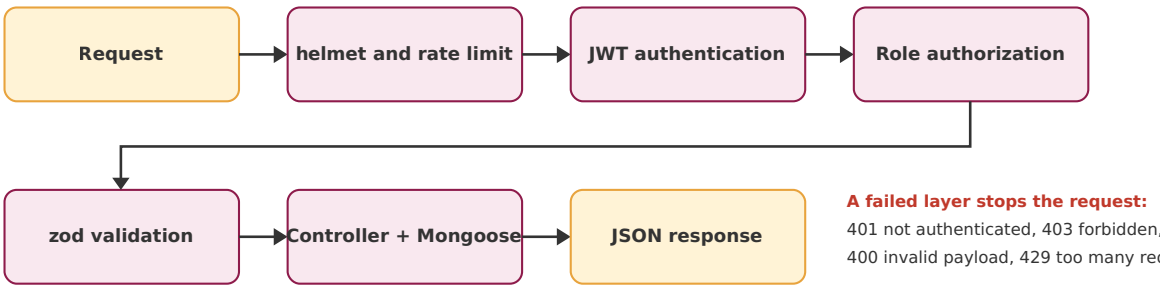


Figure 3: Layers applied to a protected request

3.4 Technology Stack

Layer	Technology	Role
Runtime	Node.js v20+	Server runtime
Framework	Express.js	REST API, modular routes and controllers
Database	MongoDB	NoSQL document database
ODM	Mongoose	Schemas, indexes and queries
Real time	Socket.IO	Live availability and desk events
Authentication	JSON Web Tokens	Stateless login and signed QR tokens
Passwords	bcryptjs	Password hashing
Validation	zod	Runtime payload validation
Hardening	helmet, express-rate-limit	Secure headers and request throttling
Scheduling	node-cron	Background no-show sweep
Frontend	React 18, Vite, TypeScript, Tailwind CSS	Single page application
Auth and push	Firebase	Required by the brief for authentication and push notifications

4. Database Design

4.1 Main Collections

MongoDB holds six collections, managed through Mongoose models.

Collection	Purpose
Members	Registered users with a role (member or manager) and hashed passwords
Spaces	Desks and meeting rooms, positioned on the floor map
Bookings	A reservation of a space for a time range, stored as 30-minute slot keys
Passes	Monthly passes held by members
Guests	Guests invited against a booking
Transactions	Charges and payments that feed the revenue reports

4.2 Relationships

Figure 4 shows how the collections relate. The relationships follow from the API, which looks up passes by user and guests by booking.

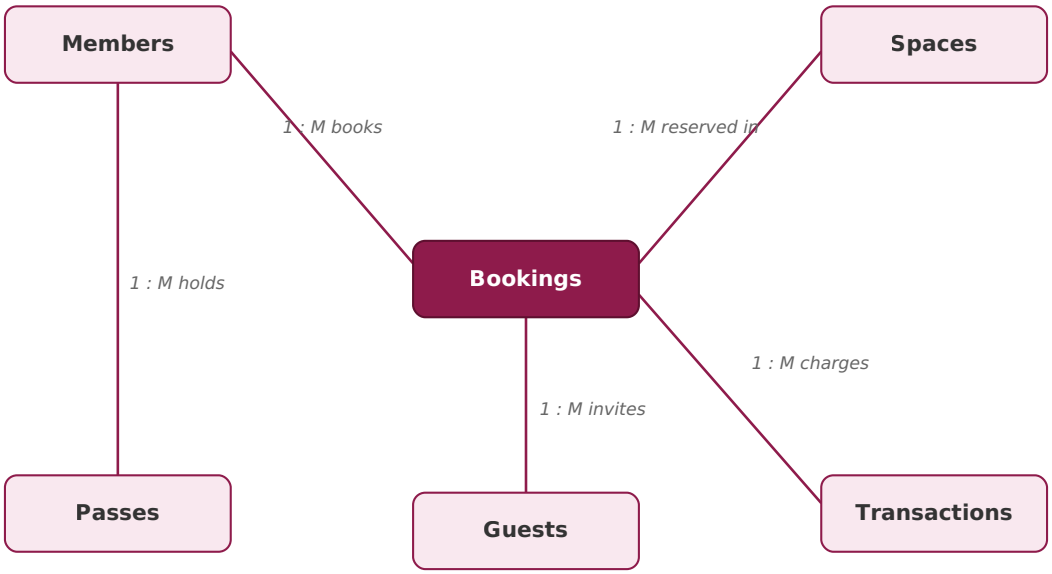


Figure 4: Relationships between the WorkHub collections

Relationship	Cardinality	Explanation
Member to Booking	1 : M	One member can make many bookings
Space to Booking	1 : M	One space can be booked many times, at different times
Member to Pass	1 : M	A member can hold passes over time
Booking to Guest	1 : M	A booking can include several guests
Booking to Transaction	1 : M	A booking can generate one or more charges

4.3 The Double-Booking Guard

The flagship design decision is where the conflict check lives. Instead of checking availability in application code, **WorkHub lets the database decide**.

1. Time is split into **30-minute intervals**. A booking from 2 PM to 5 PM covers six slots (14:00, 14:30, 15:00, 15:30, 16:00, 16:30).
2. Each Booking stores those slots as `slotKeys`.
3. A **unique partial compound index** on the Booking schema allows each space and slot to appear in only one active booking.
4. If two members book the same slot at the same moment, MongoDB accepts the first write and **rejects the second with Error 11000**. The API catches it and answers **409 Conflict (SLOT_TAKEN)**.

Because the rule is enforced by the database, it holds even when several server instances run in parallel, which a check-then-insert in application code cannot guarantee.

This sketch illustrates the idea. The field names are illustrative and it is not the project source:

```
// one active booking per space and 30-minute slot
bookingSchema.index(
  { space: 1, slotKeys: 1 },
  { unique: true, partialFilterExpression: { /* active bookings only */ } }
);
```

Evidence to attach: the real schema, as a screenshot or a code excerpt.

What to attach	Where to find it
The Booking model with `slotKeys` and the unique partial index	The Mongoose Booking schema file
The six collections in MongoDB Compass	Compass, database list

5. Backend Implementation

5.1 REST APIs

The API follows the endpoint groups of the brief: 24 endpoints in nine groups.

Group	Method	Endpoint	Purpose	Access
Auth	POST	/api/auth/register	Register	Public
Auth	POST	/api/auth/login	Login, returns a JWT	Public
Spaces	GET	/api/spaces	List spaces	Logged-in user
Spaces	GET	/api/spaces/:id	View one space	Logged-in user
Spaces	POST	/api/spaces	Create a space	Manager
Spaces	PUT	/api/spaces/:id	Update a space	Manager
Bookings	POST	/api/bookings	Create a booking	Member
Bookings	GET	/api/bookings	List bookings	Logged-in user
Bookings	GET	/api/bookings/:id	View one booking	Logged-in user
Bookings	PUT	/api/bookings/:id	Update a booking	Member (owner)
Bookings	DELETE	/api/bookings/:id	Cancel a booking	Member (owner)
Amenities	GET	/api/amenities	List amenities	Logged-in user
Amenities	GET	/api/amenities/:id	View one amenity	Logged-in user
Amenities	POST	/api/amenities	Create an amenity	Manager
Passes	POST	/api/passes	Buy a pass	Member
Passes	GET	/api/passes	List passes	Logged-in user
Passes	GET	/api/passes/user/:id	Passes of one user	Logged-in user
Guests	POST	/api/guests	Invite a guest	Member
Guests	GET	/api/guests	List guests	Logged-in user
Guests	GET	/api/guests/booking/:id	Guests of a booking	Logged-in user
QR	GET	/api/qr/check-in/:id	Verify the QR token and check in	Member
Reports	GET	/api/reports/utilization	Utilization report	Manager
Reports	GET	/api/reports/revenue	Revenue report	Manager
Notifications	POST	/api/notifications/send	Send a notification	Manager

Access levels follow the brief where it names a role (for example the manager-only space creation and reports). Where the brief is silent, the table shows the usual role split. Confirm the final levels against your route files.

Endpoints by HTTP method	Count
GET	13
POST	8
PUT	2
DELETE	1
Total	**24**

5.2 Authentication and Security

Control	Tool	Purpose
Stateless authentication	JSON Web Tokens	The server signs a token at login and verifies it on protected routes
Role-based authorization	Middleware	Separates member actions from manager actions
Password storage	bcryptjs	Passwords are stored hashed, never in plain text
Payload validation	zod	Rejects malformed requests before they reach a controller
Secure headers	helmet	Sets protective HTTP headers
Throttling	express-rate-limit	Limits repeated requests from one client

5.3 Zero Double-Booking Engine

When a member books a desk, the request follows the pipeline in Figure 5. The conflict is decided by the unique index (section 4.3), so the second of two simultaneous requests always fails cleanly.

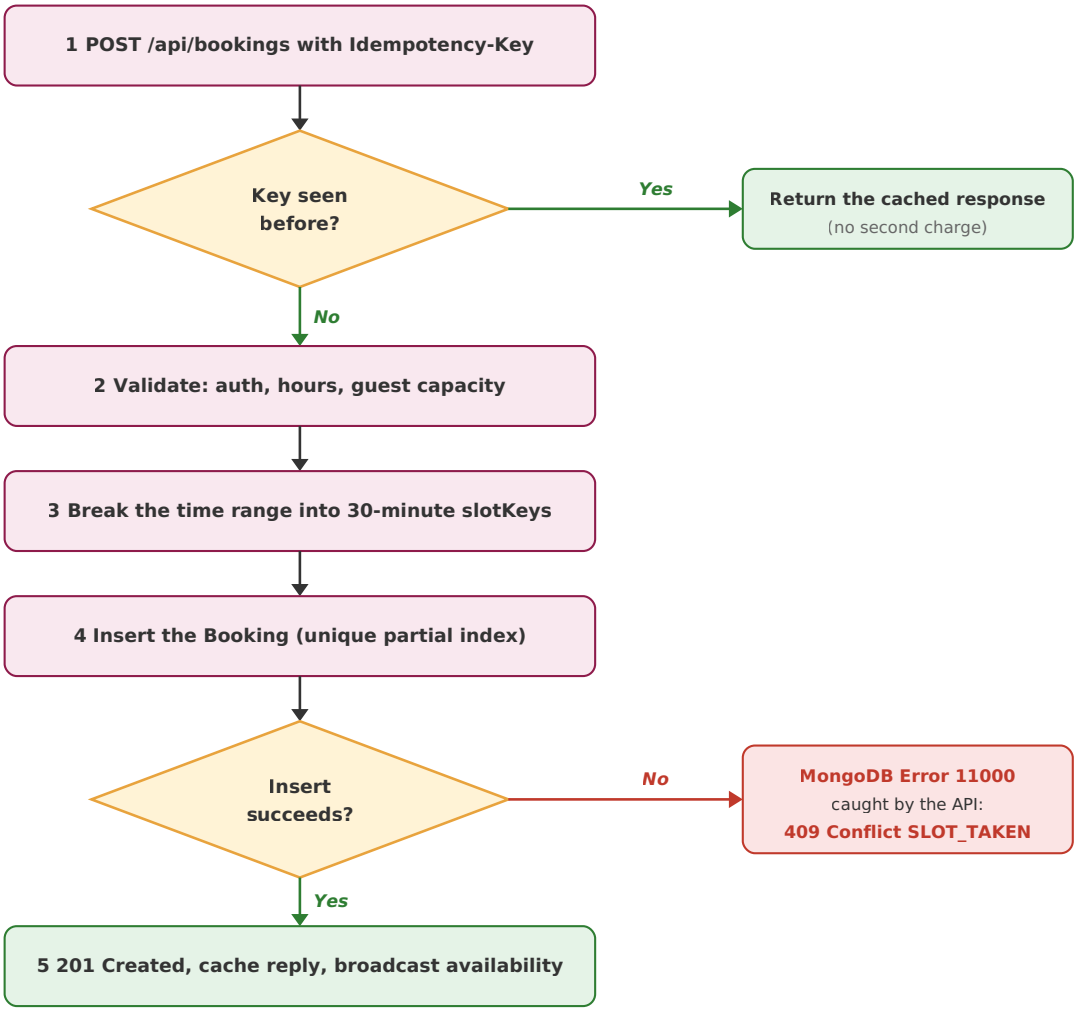


Figure 5: Booking request with idempotency and conflict handling

5.4 Idempotent Reservations

If a member's network drops during checkout, the client may send the same request again. To avoid a double charge, checkout requests carry an **`Idempotency-Key` header**. The server stores the response to the first request under that key. A later request with the same key returns the stored response instead of creating a second booking or charge.

5.5 QR Check-In and Real-Time Events

When a booking is made, the member receives a **cryptographically signed QR token** (a JWT). On arrival the member scans it at the desk. The API verifies the signature and then triggers a real-time **`desk:unlocked`** Socket.io broadcast to the member's device (Figure 6). A tampered or forged token fails the signature check and is rejected.

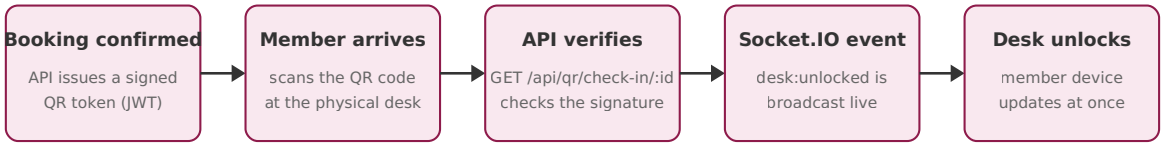


Figure 6: QR check-in flow

Socket.IO also carries the live availability updates: when a booking is created or cancelled, connected clients see the change immediately, which is what lets the floor map update as time is scrubbed.

5.6 Manager Analytics

Managers read occupancy heatmaps, utilization statistics and revenue through the reports endpoints. The figures are computed with **MongoDB aggregation pipelines**, which group and sort the booking and transaction data inside the database instead of loading it into the server.

5.7 Resilience and Edge Cases

Concern	Handling
Members who never show up	A background `node-cron` job sweeps reservations, clears no-shows and frees the desk for others
Bookings outside opening times	The API validates office operating hours
Too many guests	The API validates guest capacity limits
Simultaneous bookings	Rejected by the unique index with 409 SLOT_TAKEN
Retried requests	Replayed safely through the Idempotency-Key

5.8 Requirements Coverage

Brief requirement	How WorkHub addresses it	Status
Node.js and Express.js REST API, modular routes and controllers	Express API with route modules and controllers	Described in the project documentation
MongoDB with Mongoose for spaces, members, bookings and revenue	Six Mongoose models, including Transactions for revenue	Described in the project documentation
JWT authentication	Stateless JWT login and role-based middleware	Described in the project documentation
Firebase Auth	Required by the brief	**Confirm and attach evidence**
Socket.io real-time availability	Socket.IO server broadcasting availability and `desk:unlocked`	Described in the project documentation
Firebase push notifications for booking confirmations	Required by the brief	**Confirm and attach evidence**
Validation and role-based authorization middleware	zod validation and role middleware	Described in the project documentation
Space managers approve bookings	Required by the brief	**Confirm and attach evidence**
Amenities (printer, parking)	Endpoints in the brief	**Confirm and attach evidence**
Monthly passes and guests	Passes and Guests collections and endpoints	Described in the project documentation
Utilization and revenue reports	Aggregation pipelines behind the reports endpoints	Described in the project documentation

6. Frontend Overview

The backend is the focus of this report, but WorkHub ships with a single page application that makes the API easy to see in action. It is built with React 18, Vite, TypeScript and Tailwind CSS.

Component	What it does
FloorCanvas	An interactive, zoomable SVG floor map of the space
TimeRail	A time slider; as members scrub through time, the floor map redraws at about 60 frames per second to show availability
CommandBar	A command palette for quick navigation and booking from the keyboard
Design system	A custom glassmorphism look with frosted glass panels and mesh gradients

7. Execution Screenshots

This section presents the running system: the backend server, the database, the key API operations and the user interface.

Evidence to attach (backend): screenshots of the running API.

Screenshot	What to show
Server running	Terminal with the server and MongoDB connected messages
MongoDB collections	Compass with Members, Spaces, Bookings, Passes, Guests and Transactions
Register and login	Postman requests, with the JWT in the login response
Create a space	A manager creating a space
Create a booking	A booking request returning 201
Double booking	The same slot requested again, returning 409 SLOT_TAKEN
Idempotent replay	Two requests with the same Idempotency-Key returning the same response
Guest invite	POST /api/guests and the guests of a booking
QR check-in	The check-in request and the `desk:unlocked` event
Reports	The utilization and revenue responses

Evidence to attach (frontend): screenshots of the interface.

Screenshot	What to show
Login and register	The sign-in screen
Floor map	FloorCanvas with the TimeRail slider
Booking confirmation	A confirmed booking with its QR code
Manager dashboard	Utilization and revenue views
Command palette	CommandBar open

8. Results and Observations

8.1 Verification Plan

Run each test, record the observed result and attach a screenshot. The expected results below follow from the design.

Test	What to do	Expected result	Observed
T1 Authentication	Register, log in, then call a protected route with and without the token	JWT returned; without the token the call is rejected with 401	To record
T2 Role check	A member calls a manager endpoint such as POST /api/spaces	Rejected with 403	To record
T3 Validation	Send a malformed booking body	Rejected with 400, nothing saved	To record
T4 Booking	Create a booking for a free slot	201 Created	To record
T5 Double booking	Send two requests for the same slot at the same moment	One 201 and one 409 SLOT_TAKEN	To record
T6 Idempotency	Send the same checkout twice with one Idempotency-Key	Identical responses and a single booking	To record
T7 Real time	Open two clients and book from one	The other client sees the availability change	To record
T8 QR check-in	Scan a valid token, then a tampered one	Valid token fires `desk:unlocked`; tampered token rejected	To record
T9 No-show sweep	Leave a booking unattended past its time	The cron job clears it and the desk becomes free	To record
T10 Reports	A manager requests utilization and revenue	Aggregated figures returned; members are refused	To record
T11 Rate limit	Send requests faster than the limit	Excess requests get 429	To record

> **Evidence to attach:** fill in the Observed column for T1 to T11 and attach the screenshot for each.

8.2 Key Observations

1. **Concurrency safety lives in the database.** The unique partial index rejects the second write, so the guarantee does not depend on application timing.
2. **Retries are safe.** The Idempotency-Key lets the server return the first response instead of repeating a booking or charge.
3. **Check-in cannot be forged.** The QR token is signed, and the server verifies the signature before unlocking.
4. **Analytics run in the database.** Aggregation pipelines keep reporting fast and the server light.
5. **Stale reservations clear themselves.** The cron sweep returns no-show desks to the pool.

8.3 Assumptions and Limits

- Bookings are made in **30-minute slots**, so times are rounded to slot boundaries.
- The no-show sweep runs periodically, so a freed desk appears after the next sweep rather than the instant a booking lapses.
- How long a cached response is kept decides how long a retry with the same Idempotency-Key remains safe.
- The unique index only protects data once it is built, so it must exist in every environment where the API runs.
- Firebase features depend on external credentials and network access.

9. Conclusion and Future Scope

9.1 Conclusion

WorkHub demonstrates a complete backend for a co-working space: authentication with roles, space and booking management, passes, guests and amenities, QR check-in, real-time availability and manager analytics. Its central strength is that the hard rules are enforced on the server: a unique partial index prevents double booking, an Idempotency-Key makes retries safe, signed QR tokens secure check-in, and a cron job keeps availability honest.

The project also shows how the parts of a backend cooperate:

Concept	Learning	Concept	Learning
Express.js	Modular routes and controllers	**Socket.IO**	Live availability and events
JWT	Stateless authentication and signed QR tokens	**Aggregation**	Analytics inside the database
Mongoose	Schemas, relations and indexes	**Idempotency**	Safe retries
Middleware	Reusable security and validation	**Unique index**	Database-level concurrency control

9.2 Deliverables

Deliverable in the brief	Status
REST API with all CRUD operations	Described in sections 5.1 to 5.7
WebSocket server for real-time availability	Socket.IO, described in section 5.5
API documentation (Swagger or Postman collection)	**Attach the link or the collection**
GitHub repository with README and setup instructions	**Attach the repository link**
Bonus: deployment on Render or Heroku	Optional, **attach the live URL if deployed**

9.3 Future Scope

Enhancement	Note
Access control integration API	Optional feature in the brief
Happy hours pricing API	Optional feature in the brief
Team room bookings API	Optional feature in the brief
Cloud deployment on Render or Heroku	Bonus deliverable in the brief
Email and push reminders before a booking	Proposal, builds on the notification endpoint
Automated tests for the concurrency and idempotency paths	Proposal, protects the core guarantees